

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA : Raka Yulia Vegora

NIM : 109082500201

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Sorting (pengurutan) adalah proses menyusun sekumpulan data ke dalam urutan tertentu, baik secara **ascending** (dari kecil ke besar) maupun **descending** (dari besar ke kecil). Pengurutan merupakan salah satu operasi dasar yang sangat penting dalam ilmu komputer karena banyak algoritma lain seperti pencarian biner (binary search) membutuhkan data yang sudah terurut sebagai syarat utamanya.

Terdapat beberapa algoritma pengurutan yang umum dipelajari, di antaranya adalah **Bubble Sort**, **Selection Sort**, **Insertion Sort**, dan **Merge Sort**. Setiap algoritma memiliki cara kerja, kelebihan, dan kekurangan masing-masing, serta karakteristik kompleksitas waktu yang berbeda-beda.

Bubble Sort adalah algoritma pengurutan paling sederhana yang bekerja dengan cara membandingkan dua elemen yang berdekatan secara berulang, kemudian menukarnya jika urutannya salah. Proses ini diulang hingga tidak ada lagi pertukaran yang perlu dilakukan. Algoritma ini memiliki kompleksitas waktu $O(n^2)$ pada kasus rata-rata dan terburuk, sehingga kurang efisien untuk data dalam jumlah besar.

Selection Sort bekerja dengan cara mencari elemen terkecil (atau terbesar) dari data yang belum terurut, kemudian menempatkannya pada posisi yang sesuai. Proses ini diulang sebanyak $n-1$ kali hingga seluruh data terurut. Algoritma ini juga memiliki kompleksitas waktu $O(n^2)$ namun melakukan lebih sedikit operasi penukaran dibandingkan Bubble Sort.

Insertion Sort bekerja dengan cara mengambil satu elemen dari data yang belum terurut, kemudian menyisipkannya ke posisi yang tepat pada bagian data yang sudah terurut. Algoritma ini mirip dengan cara manusia mengurutkan kartu secara manual. Insertion Sort memiliki kompleksitas waktu $O(n^2)$ pada kasus terburuk, namun sangat efisien untuk data yang hampir terurut karena kompleksitas terbaiknya adalah $O(n)$.

Merge Sort adalah algoritma pengurutan yang menggunakan pendekatan **divide and conquer**, yaitu membagi data menjadi dua bagian yang lebih kecil, mengurutkan masing-masing bagian secara rekursif, kemudian menggabungkannya kembali menjadi satu urutan yang benar. Algoritma ini memiliki kompleksitas waktu $O(n \log n)$ pada semua kasus, sehingga lebih efisien untuk data dalam jumlah besar.

Selain pengurutan data numerik, algoritma sorting juga dapat diterapkan pada data bertipe string maupun struct dengan menyesuaikan kriteria perbandingan yang digunakan. Pada bahasa pemrograman Go, pengurutan dapat dilakukan secara manual menggunakan algoritma di atas atau menggunakan package bawaan sort yang sudah disediakan oleh standard library.

B. Guided

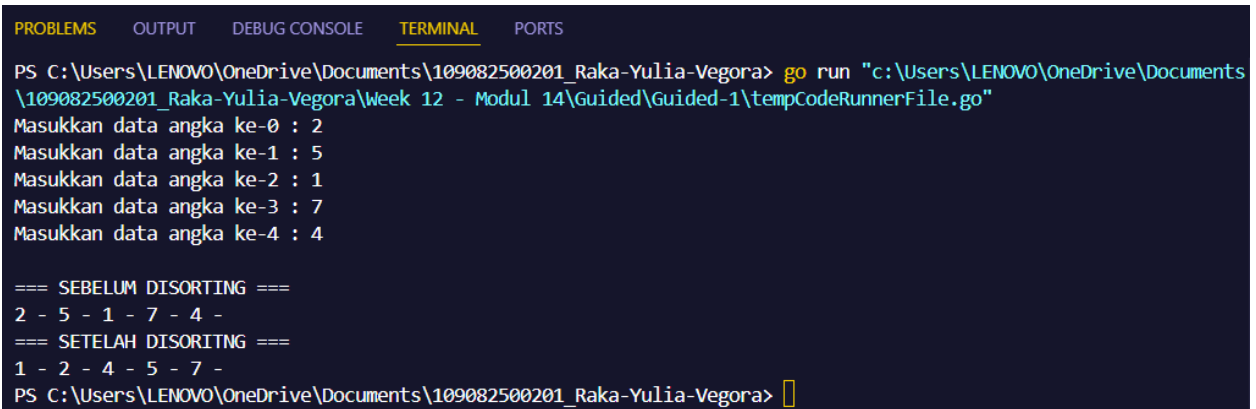
1. SelSortArray.go

```
1 package main
2
3 import "fmt"
4
5 func selecetionSortArray(angka *[5]int){
6     var idx_min, i, j int
7     for i = 0; i < len(angka) - 1; i++ { //Loop Luar (iterasi sortingnya)
8         idx_min = i
9         for j = i + 1; j < len(angka); j++ { //Loop dalam (mencari nilai ekstrim)
10             if angka[j] <= angka[idx_min] { //sorting terkecil ke terbesar
11                 idx_min = j
12             }
13         }
14         if idx_min != i {
15             angka[i], angka[idx_min] = angka[idx_min], angka[i]
16         }
17     }
18 }
19
20 func main(){
21     var arrAngka [5]int
22
23     for i := 0; i < len(arrAngka); i++ {
24         fmt.Printf("Masukkan data angka ke-%d : ", i)
25         fmt.Scan(&arrAngka[i])
26     }
27     fmt.Println()
28
29     fmt.Println("=== SEBELUM DISORTING ===")
30     for i := 0; i < len(arrAngka); i++ {
31         fmt.Print(arrAngka[i], " - ")
32     }
33
34     fmt.Println("\n=== SETELAH DISORITNG ===")
35     selecetionSortArray(&arrAngka)
36     for i := 0; i < len(arrAngka); i++ {
37         fmt.Print(arrAngka[i], " - ")
38     }
39 }
```

Penjelasan :

Program ini digunakan untuk mengimplementasikan algoritma **Selection Sort** untuk mengurutkan array berisi 5 bilangan integer dari terkecil ke terbesar. Fungsi `selecetionSortArray` bekerja dengan dua loop bersarang — loop luar menentukan posisi saat ini, sedangkan loop dalam mencari elemen terkecil yang tersisa, lalu menukarnya ke posisi yang tepat. Di fungsi `main`, pengguna diminta memasukkan 5 angka, kemudian program menampilkan array **sebelum** dan **sesudah** proses pengurutan dilakukan.

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Guided\Guided-1\tempCodeRunnerFile.go"
Masukkan data angka ke-0 : 2
Masukkan data angka ke-1 : 5
Masukkan data angka ke-2 : 1
Masukkan data angka ke-3 : 7
Masukkan data angka ke-4 : 4

=== SEBELUM DISORTING ===
2 - 5 - 1 - 7 - 4 -
=== SETELAH DISORITNG ===
1 - 2 - 4 - 5 - 7 -
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> 
```

2. SelSortStruct.go

```
1  package main
2
3  import "fmt"
4
5  type mahasiswa struct {
6      nama, nim string
7      IPK      float64
8  }
9
10 func SelectionSortArray(sMHS *[5]mahasiswa) {
11     var idx_min, i, j int
12     for i = 0; i < len(sMHS)-1; i++ { //Loop Luar
13         idx_min = i
14         for j = i + 1; j < len(sMHS); j++ { //Loop dalam mencari nilai ekstrim
15             if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil ke terbesar
16                 idx_min = j
17             }
18         }
19         if idx_min != i {
20             sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
21         }
22     }
23 }
24
25 func main() {
26     var structMHS [5]mahasiswa
27
28     for i := 0; i < len(structMHS); i++ {
29         fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
30         fmt.Print("Nama : ")
31         fmt.Scan(&structMHS[i].nama)
32         fmt.Print("NIM : ")
33         fmt.Scan(&structMHS[i].nim)
34         fmt.Print("IPK : ")
35         fmt.Scan(&structMHS[i].IPK)
36     }
37     fmt.Println()
38
39     fmt.Println("=== SEBELUM SORITNG === ")
40     for i := 0; i < len(structMHS); i++ {
41         fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
42         fmt.Printf("Nama : %s\n", structMHS[i].nama)
43         fmt.Printf("NIM : %s\n", structMHS[i].nim)
44         fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
45     }
46     fmt.Println("=====")
47     fmt.Println()
48
49     SelectionSortArray(&structMHS)
50     fmt.Println("=== SETELAH SORITNG === ")
51     for i := 0; i < len(structMHS); i++ {
52         fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
53         fmt.Printf("Nama : %s\n", structMHS[i].nama)
54         fmt.Printf("NIM : %s\n", structMHS[i].nim)
55         fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
56     }
57     fmt.Println("=====")
58     fmt.Println()
59 }
60
```

Penjelasan :

Program ini adalah **sistem manajemen data buku** yang mampu menyimpan hingga 7919 data. Program menerima input data buku (id, judul, penulis, penerbit, eksemplar, tahun, rating), lalu secara otomatis mencari buku terfavorit berdasarkan rating tertinggi, mengurutkan data menggunakan **insertion sort**, menampilkan 5 buku rating teratas, serta menyediakan fitur pencarian buku berdasarkan rating menggunakan **binary search**.

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Nama : Raka
NIM : 101
IPK : 3.80
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : Sigit
NIM : 102
IPK : 3.85
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : Dharent
NIM : 103
IPK : 3.70
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : Habil
NIM : 104
IPK : 3.80
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : Danis
NIM : 105
IPK : 3.87

=== SEBELUM SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : Raka
NIM : 101
IPK : 3.80
--- Data Mahasiswa Ke-1 ---
Nama : Sigit
NIM : 102
IPK : 3.85
--- Data Mahasiswa Ke-2 ---
Nama : Dharent
NIM : 103
IPK : 3.70
--- Data Mahasiswa Ke-3 ---
Nama : Habil
NIM : 104
IPK : 3.80
--- Data Mahasiswa Ke-4 ---
Nama : Danis
NIM : 105
IPK : 3.87
=====
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
=== SETELAH SORITNG ===  
--- Data Mahasiswa Ke-0 ---  
Nama : Dharent  
NIM : 103  
IPK : 3.70  
--- Data Mahasiswa Ke-1 ---  
Nama : Raka  
NIM : 101  
IPK : 3.80  
--- Data Mahasiswa Ke-2 ---  
Nama : Habil  
NIM : 104  
IPK : 3.80  
--- Data Mahasiswa Ke-3 ---  
Nama : Sigit  
NIM : 102  
IPK : 3.85  
--- Data Mahasiswa Ke-4 ---  
Nama : Danis  
NIM : 105  
IPK : 3.87  
=====
```

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>

3. InsertionSort.go

```
1  package main
2
3  import "fmt"
4
5  const nMax int = 4321
6
7  type arrInt [nMax]int
8
9  func insertionSort(T *arrInt, n int) {
10     /* I.S. terdefinisi array T yang berisi n bilangan bulat
11        F.S. array T terurut secara mengecil atau descending dengan INSERTION SORT */
12
13     var temp, i, j int
14
15     i = 1
16     for i <= n-1 {
17         j = i
18         temp = T[j]
19
20         for j > 0 && temp > T[j-1] {
21             T[j] = T[j-1]
22             j = j - 1
23         }
24
25         T[j] = temp
26         i = i + 1
27     }
28 }
29
30 func main() {
31     var data arrInt
32     var n int
33
34     fmt.Scan(&n)
35
36     for i := 0; i < n; i++ {
37         fmt.Scan(&data[i])
38     }
39
40     insertionSort(&data, n)
41
42     for i := 0; i < n; i++ {
43         fmt.Print(data[i], " ")
44     }
45 }
46
```


Penjelasan :

Program ini digunakan untuk mengimplementasikan algoritma **Insertion Sort** untuk mengurutkan array bilangan bulat secara **descending** (dari terbesar ke terkecil). Fungsi `insertionSort` bekerja dengan mengambil elemen satu per satu mulai dari indeks ke-1, menyimpannya ke variabel `temp`, lalu menggeser elemen-elemen sebelumnya ke kanan selama nilainya lebih kecil dari `temp`. Elemen kemudian disisipkan ke posisi yang tepat. Di fungsi `main`, pengguna memasukkan jumlah data `n` beserta nilai-nilainya, lalu fungsi `insertionSort` dipanggil dan hasil pengurutan ditampilkan ke layar.

Output :



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Guided\Guided-2\insertionSort.go"
5
2 5 1 10 6
10 6 5 2 1
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

4. SearchingStruct.go

```
1  package main
2
3  import "fmt"
4
5  const nMax int = 100
6
7  type arrString [nMax]string
8
9  func insertionSortString(T *arrString, n int) {
10     var temp string
11     var i, j int
12
13     i = 1
14     for i <= n-1 {
15         j = i
16         temp = T[j]
17
18         for j > 0 && temp < T[j-1] {
19             T[j] = T[j-1]
20             j = j - 1
21         }
22
23         T[j] = temp
24         i = i + 1
25     }
26 }
27
28 func main() {
29     var data arrString
30     var n int
31
32     fmt.Scan(&n)
33
34     for i := 0; i < n; i++ {
35         fmt.Scan(&data[i])
36     }
37
38     insertionSortString(&data, n)
39
40     for i := 0; i < n; i++ {
41         fmt.Print(data[i], " ")
42     }
43 }
```

Penjelasan :

Program ini digunakan untuk mengimplementasikan algoritma **Insertion Sort** untuk mengurutkan array **string** secara **ascending** (dari terkecil ke terbesar secara alfabetis). Fungsi `insertionSortString` bekerja dengan mengambil elemen string satu per satu mulai indeks ke-1, menyimpannya ke variabel `temp`, lalu menggeser elemen sebelumnya ke kanan selama nilainya lebih besar dari `temp`. String kemudian disisipkan ke posisi yang sesuai. Di fungsi `main`, pengguna memasukkan jumlah data `n` beserta nilai string-nya, lalu fungsi `insertionSortString` dipanggil dan hasil pengurutan ditampilkan ke layar. Perbedaan utama dengan versi sebelumnya adalah tipe data yang digunakan berupa **string** dan urutan sortingnya **ascending**.

Output :



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Guided\Guided-3\insertionSortString.go"
5
Apel Strawberry Mangga Durian Pisang
Apel Durian Mangga Pisang Strawberry
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

Jawaban :

```
1 package main
2
3 import "fmt"
4
5 func selectionSort(arr []int) {
6     n := len(arr)
7     for i := 0; i < n-1; i++ {
8         minIdx := i
9         for j := i + 1; j < n; j++ {
10             if arr[j] < arr[minIdx] {
11                 minIdx = j
12             }
13         }
14         arr[i], arr[minIdx] = arr[minIdx], arr[i]
15     }
16 }
17
18 func main() {
19     var n int
20     fmt.Scan(&n)
21
22     for i := 0; i < n; i++ {
23         var m int
24         fmt.Scan(&m)
25
26         rumah := make([]int, m)
27         for j := 0; j < m; j++ {
28             fmt.Scan(&rumah[j])
29         }
30
31         selectionSort(rumah)
32
33         fmt.Printf("Daerah %d: ", i+1)
34         for j := 0; j < m; j++ {
35             if j > 0 {
36                 fmt.Print(" ")
37             }
38             fmt.Print(rumah[j])
39         }
40         fmt.Println()
41     }
42 }
```

Penjelasan :

Program ini digunakan untuk mengimplementasikan algoritma **Selection Sort** untuk mengurutkan data dari beberapa daerah secara **ascending**. Fungsi `selectionSort` menerima slice integer dan mengurutkannya dengan mencari nilai terkecil menggunakan dua loop bersarang, lalu menukarnya ke posisi yang tepat. Di fungsi `main`, pengguna

memasukkan jumlah daerah n , kemudian untuk setiap daerah diminta memasukkan jumlah rumah m beserta nilai-nilainya. Setiap data daerah disimpan dalam slice rumah, diurutkan menggunakan selectionSort, lalu ditampilkan per daerah dengan format "**Daerah 1: ...**", "**Daerah 2: ...**" dan seterusnya.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Unguided\Unguided-no1\rumahkerabat.go"
3
5 2 1 7 9 13
Daerah 1: 1 2 7 9 13
6 189 15 27 39 75 133
Daerah 2: 15 27 39 75 133 189
3 4 9 1
Daerah 3: 1 4 9
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> |
```

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban :

```

1  package main
2
3  import "fmt"
4
5  func bubbleSortAsc(arr []int) {
6      n := len(arr)
7      for i := 0; i < n-1; i++ {
8          for j := 0; j < n-i-1; j++ {
9              if arr[j] > arr[j+1] {
10                 arr[j], arr[j+1] = arr[j+1], arr[j]
11             }
12         }
13     }
14 }
15
16 func bubbleSortDesc(arr []int) {
17     n := len(arr)
18     for i := 0; i < n-1; i++ {
19         for j := 0; j < n-i-1; j++ {
20             if arr[j] < arr[j+1] {
21                 arr[j], arr[j+1] = arr[j+1], arr[j]
22             }
23         }
24     }
25 }
26
27 func main() {
28     var n int
29     fmt.Scan(&n)
30
31     for i := 0; i < n; i++ {
32         var count int
33         fmt.Scan(&count)
34
35         odds := []int{}
36         evens := []int{}
37
38         for j := 0; j < count; j++ {
39             var num int
40             fmt.Scan(&num)
41             if num%2 != 0 {
42                 odds = append(odds, num)
43             } else {
44                 evens = append(evens, num)
45             }
46         }
47
48         // Ganjil urut naik
49         bubbleSortAsc(odds)
50
51         // Genap urut turun
52         bubbleSortDesc(evens)
53
54         // Cetak dengan Label Daerah
55         fmt.Printf("Daerah %d:", i+1)
56         for _, v := range odds {
57             fmt.Printf(" %d", v)
58         }
59         for _, v := range evens {
60             fmt.Printf(" %d", v)
61         }
62         fmt.Println()
63     }
64 }

```

Penjelasan :

Program ini digunakan untuk mengimplementasikan algoritma **Bubble Sort** dengan dua mode pengurutan berbeda. Terdapat dua fungsi sorting: `bubbleSortAsc` untuk mengurutkan bilangan **ganjil secara ascending** dan `bubbleSortDesc` untuk mengurutkan bilangan **genap secara descending**, keduanya bekerja dengan membandingkan dan menukar elemen bersebelahan secara berulang. Di fungsi main, pengguna memasukkan jumlah daerah `n`, lalu untuk setiap daerah diminta memasukkan sejumlah angka yang kemudian dipisahkan ke dalam dua slice terpisah — odds untuk bilangan ganjil dan evens untuk bilangan genap. Setelah diurutkan sesuai jenisnya, hasil dari kedua slice digabung dan ditampilkan per daerah dengan format "**Daerah 1: ...**", "**Daerah 2: ...**" dan seterusnya.

Output :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Unguided\Unguided-no2\rumahKerabatganjilgenap.go"
3
5 2 1 7 9 13
Daerah 1: 1 7 9 13 2
6 189 15 27 39 75 133
Daerah 2: 15 27 39 75 133 189
3 4 9 1
Daerah 3: 1 9 4
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> 
```

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

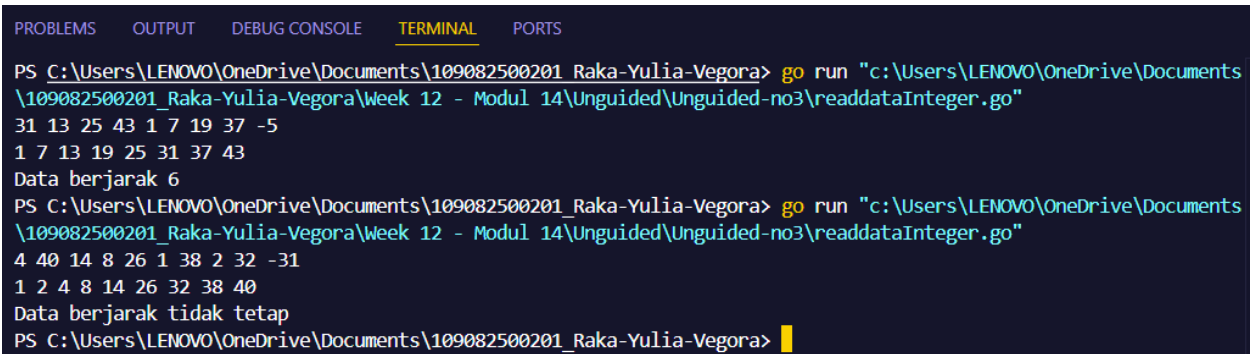
Jawaban :

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     arr := make([]int, 0)
7
8     // Baca input sampai bilangan negatif
9     for {
10         var x int
11         fmt.Scan(&x)
12         if x < 0 {
13             break
14         }
15         arr = append(arr, x)
16     }
17
18     // Insertion Sort ascending
19     for i := 1; i < len(arr); i++ {
20         key := arr[i]
21         j := i - 1
22         for j >= 0 && arr[j] > key {
23             arr[j+1] = arr[j]
24             j--
25         }
26         arr[j+1] = key
27     }
28
29     // Cetak array terurut
30     for i := 0; i < len(arr); i++ {
31         if i > 0 {
32             fmt.Print(" ")
33         }
34         fmt.Print(arr[i])
35     }
36     fmt.Println()
37
38     // Cek jarak
39     tetap := true
40     jarak := 0
41     if len(arr) >= 2 {
42         jarak = arr[1] - arr[0]
43         for i := 2; i < len(arr); i++ {
44             if arr[i]-arr[i-1] != jarak {
45                 tetap = false
46                 break
47             }
48         }
49     }
50
51     if tetap {
52         fmt.Println("Data berjarak", jarak)
53     } else {
54         fmt.Println("Data berjarak tidak tetap")
55     }
56 }
```


Penjelasan :

Program ini digunakan untuk membaca input bilangan bulat secara terus-menerus hingga pengguna memasukkan **bilangan negatif** sebagai penanda berhenti. Data yang valid disimpan ke dalam slice arr, lalu diurutkan secara **ascending** menggunakan algoritma **Insertion Sort**. Setelah diurutkan dan ditampilkan, program melakukan pengecekan apakah selisih antar elemen berurutan selalu sama (konstan). Jika selisihnya tetap, program mencetak "**Data berjarak**" beserta nilai jaraknya, dan jika tidak konsisten mencetak "**Data berjarak tidak tetap**".

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Unguided\Unguided-no3\readdataInteger.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Unguided\Unguided-no3\readdataInteger.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Jawaban :

```

1 package main
2 import "fmt"
3
4 const nMax = 7919
5
6 type Buku struct {
7     id, judul, penulis, penerbit string
8     eksemplar, tahun, rating      int
9 }
10
11 type DaftarBuku [1 + nMax]Buku
12
13 var Pustaka DaftarBuku
14 var nPustaka int
15
16 func cetakGaris() {
17     fmt.Println("=====")
18 }
19
20 func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
21     fmt.Scan(n)
22     for i := 1; i <= *n; i++ {
23         fmt.Scan(&pustaka[i].id)
24         fmt.Scan(&pustaka[i].judul)
25         fmt.Scan(&pustaka[i].penulis)
26         fmt.Scan(&pustaka[i].penerbit)
27         fmt.Scan(&pustaka[i].eksemplar)
28         fmt.Scan(&pustaka[i].tahun)
29         fmt.Scan(&pustaka[i].rating)
30     }
31 }
32
33 func CetakTerfavorit(pustaka DaftarBuku, n int) {
34     terfavorit := pustaka[1]
35     for i := 2; i <= n; i++ {
36         if pustaka[i].rating > terfavorit.rating {
37             terfavorit = pustaka[i]
38         }
39     }
40     cetakGaris()
41     fmt.Println("          ★ BUKU TERFAVORIT ★")
42     cetakGaris()
43     fmt.Printf(" ID      : %s\n", terfavorit.id)
44     fmt.Printf(" Judul   : %s\n", terfavorit.judul)
45     fmt.Printf(" Penulis : %s\n", terfavorit.penulis)
46     fmt.Printf(" Penerbit : %s\n", terfavorit.penerbit)
47     fmt.Printf(" Tahun   : %d\n", terfavorit.tahun)
48     fmt.Printf(" Rating  : %d/10\n", terfavorit.rating)
49 }
50
51 func UrutBuku(pustaka *DaftarBuku, n int) {
52     for i := 2; i <= n; i++ {
53         key := pustaka[i]
54         j := i - 1
55         for j >= 1 && pustaka[j].rating < key.rating {
56             pustaka[j+1] = pustaka[j]
57             j--
58         }
59         pustaka[j+1] = key
60     }
61 }
62
63 func Cetak5Terbaru(pustaka DaftarBuku, n int) {
64     batas := n
65     if batas > 5 {
66         batas = 5
67     }
68     cetakGaris()
69     fmt.Println("          ★ 5 BUKU RATING TERTINGGI ★")
70     cetakGaris()
71     for i := 1; i <= batas; i++ {
72         fmt.Printf(" %d. %-20s\n", i, pustaka[i].judul)
73         fmt.Printf("      Penulis : %s\n", pustaka[i].penulis)
74         fmt.Printf("      Penerbit : %s\n", pustaka[i].penerbit)
75         fmt.Printf("      Rating  : %d/10\n", pustaka[i].rating)
76         if i < batas {
77             fmt.Println(" -----")
78         }
79     }
80 }
81
82 func CariBuku(pustaka DaftarBuku, n int, r int) {
83     lo, hi := 1, n
84     idx := -1
85     for lo <= hi {
86         mid := (lo + hi) / 2
87         if pustaka[mid].rating == r {
88             idx = mid
89             break
90         } else if pustaka[mid].rating > r {
91             lo = mid + 1
92         } else {
93             hi = mid - 1
94         }
95     }
96     cetakGaris()
97     fmt.Printf("          ★ HASIL PENCARIAN RATING %d ★\n", r)
98     cetakGaris()
99     if idx == -1 {
100         fmt.Printf(" [!] Tidak ada buku dengan rating %d\n", r)
101     } else {
102         fmt.Printf(" ID      : %s\n", pustaka[idx].id)
103         fmt.Printf(" Judul   : %s\n", pustaka[idx].judul)
104         fmt.Printf(" Penulis : %s\n", pustaka[idx].penulis)
105         fmt.Printf(" Penerbit : %s\n", pustaka[idx].penerbit)
106         fmt.Printf(" Tahun   : %d\n", pustaka[idx].tahun)
107         fmt.Printf(" Eksemplar : %d\n", pustaka[idx].eksemplar)
108         fmt.Printf(" Rating  : %d/10\n", pustaka[idx].rating)
109     }
110     cetakGaris()
111 }
112
113 func main() {
114     DaftarkanBuku(&Pustaka, &nPustaka)
115     CetakTerfavorit(Pustaka, nPustaka)
116     UrutBuku(&Pustaka, nPustaka)
117     Cetak5Terbaru(Pustaka, nPustaka)
118
119     var r int
120     fmt.Scan(&r)
121     CariBuku(Pustaka, nPustaka, r)
122 }

```

Penjelasan :

Program ini digunakan untuk mengelola **daftar pustaka (perpustakaan)** dengan berbagai fitur. Struct Buku menyimpan data buku berupa id, judul, penulis, penerbit, eksemplar, tahun, dan rating. Program memiliki lima fungsi utama: DaftarkanBuku untuk menginput data buku, CetakTerfavorit untuk menampilkan buku dengan **rating tertinggi**, UrutBuku untuk mengurutkan buku berdasarkan rating secara **ascending** menggunakan Insertion Sort, Cetak5Terbaru untuk menampilkan **5 buku dengan rating tertinggi** setelah diurutkan, dan CariBuku untuk mencari buku berdasarkan rating menggunakan algoritma **Binary Search**. Di fungsi main, semua fungsi tersebut dipanggil secara berurutan — mulai dari pendaftaran buku, pencetakan favorit, pengurutan, tampilan 5 terbaik, hingga pencarian buku berdasarkan rating yang diinputkan pengguna.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

\109082500201_Raka-Yulia-Vegora\Week 12 - Modul 14\Unguided\Unguided-no4\dataBuku.go"
5
B001 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 3 2005 9
B002 Bumi_Manusia Pramoedya Lentera_Dipantara 2 1980 8
B003 Negeri_5_Menara Ahmad_Fuadi Gramedia 5 2009 7
B004 Perahu_Kertas Dee_Lestari Bentang_Pustaka 4 2009 9
B005 Ayat-Ayat_Cinta Habiburrahman Republika 6 2004 6
=====
      ★ BUKU TERFAVORIT ★
=====
ID       : B001
Judul    : Laskar_Pelangi
Penulis  : Andrea_Hirata
Penerbit : Bentang_Pustaka
Tahun    : 2005
Rating   : 9/10
=====
      ★ 5 BUKU RATING TERTINGGI ★
=====
1. Laskar_Pelangi
   Penulis  : Andrea_Hirata
   Penerbit : Bentang_Pustaka
   Rating   : 9/10
-----
2. Perahu_Kertas
   Penulis  : Dee_Lestari
   Penerbit : Bentang_Pustaka
   Rating   : 9/10
-----
3. Bumi_Manusia
   Penulis  : Pramoedya
   Penerbit : Lentera_Dipantara
   Rating   : 8/10
-----
4. Negeri_5_Menara
   Penulis  : Ahmad_Fuadi
   Penerbit : Gramedia
   Rating   : 7/10
-----
5. Ayat-Ayat_Cinta
   Penulis  : Habiburrahman
   Penerbit : Republika
   Rating   : 6/10
```

```
8
=====
★  HASIL  PENCARIAN  RATING  8  ★
=====
ID       : B002
Judul    : Bumi_Manusia
Penulis  : Pramoedya
Penerbit : Lentera_Dipantara
Tahun    : 1980
Eksemplar : 2
Rating   : 8/10
=====
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora>
```

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting merupakan teknik penting dalam pengolahan data untuk menyusun data secara teratur, baik secara ascending maupun descending. Berbagai metode sorting seperti Selection Sort, Bubble Sort, dan Insertion Sort memiliki cara kerja yang berbeda-beda, namun tujuan utamanya sama yaitu menghasilkan data yang terurut.

Dari hasil implementasi program, diketahui bahwa Selection Sort bekerja dengan mencari nilai minimum untuk ditempatkan pada posisi yang tepat, sedangkan Insertion Sort lebih efektif untuk data yang hampir terurut karena melakukan penyisipan elemen pada posisi yang sesuai. Bubble Sort digunakan untuk membandingkan elemen berdekatan secara berulang, meskipun kurang efisien untuk data berukuran besar.

Selain itu, algoritma sorting dapat diterapkan pada berbagai tipe data seperti integer dan string, serta dapat dikombinasikan dengan proses lain seperti pencarian (searching) dan pengolahan struct. Melalui praktikum ini, pemahaman terhadap konsep sorting dan penerapannya dalam pemrograman, khususnya menggunakan bahasa Go, menjadi lebih baik.

Secara keseluruhan, praktikum ini membantu meningkatkan kemampuan dalam merancang algoritma, memahami logika pemrograman, serta mengimplementasikan berbagai metode pengurutan sesuai kebutuhan kasus.